LangChain in plain English

Written for a cloud / infra person, not a developer · a 10-minute read before an interview

1 What problem does it solve?

In Chapter 01 we built RAG **by hand**: our own embed code, cosine maths, sorting and top-K. Real projects use **many vendors** (PDF readers, OpenAI, Qdrant, OpenRouter), each with a different SDK. LangChain gives every job one standard plug, so you snap the pieces together and can **swap any vendor by changing one line.**

🔧 Like a **Kubernetes CSI driver**: the Pod asks for "a volume", and EBS, Azure Disk or NFS sits underneath. Or Terraform core + providers.

It IS ✅

a **toolbox / framework**: loaders, splitters, embedding and vector-store wrappers, plus the **|** pipe to join steps (with streaming and batching free).

It is NOT ❌

not RAG (RAG is the design) · **not a database** (Qdrant stores) · **not a model** (OpenAI/OpenRouter do the AI)

2 The 8 building blocks

Prepare once indexing, a batch job

Loader → Splitter → Embeddings → Vector store

Answer every time querying, the live path

Retriever → Prompt → Chat model → Output parser

3 The whole chain in one line (LCEL)

```
{context: retriever | format_docs, question: passthrough} | prompt | llm | StrOutputParser()
```

🔧 It's a **Unix pipe**: `cat log | grep 500 | sort | uniq -c`. The output of the left feeds the right.

Quantization 🗜

Store vectors at lower precision: float32 **6.1 GB** → int8 **1.5 GB** → binary **0.19 GB** (1M chunks). Like a tape with only **feet** marked: you lose the inches, not the answer.

Debug rule 🔍

Wrong answer? Walk the stages: **retrieved?** → context → prompt → LLM. The "client dinner" question failed because **expense_policy.pdf was never indexed!**

Getting data in

Each term has a one-line meaning plus 🔧 an infra analogy

① Core objects

Document: **page_content** (the text) + **metadata** (source, page, any key).
🔧 A log line + its labels (pod, namespace).

Metadata: becomes the **payload** in Qdrant. Used to filter and to cite "file X, page 3".

Runnable: anything with `.invoke()` (retriever, prompt, LLM, parser), so it can be piped.
🔧 Like a container that follows a standard interface.

langchain-core: the hub package. Others (-community, -openai, -qdrant, -openrouter) plug in.
🔧 Terraform core + providers.

② Loaders

Document loader: reads a source and returns **list[Document]**. Same output for every file type.
🔧 Like Fluent Bit inputs: many sources, one format.

TextLoader: 1 file → 1 Document. Keeps the Markdown **headings** (good for splitting).

PyPDF / PyMuPDF: 1 Document **per page**. PyMuPDF is fastest but AGPL licence.

OCR: reads **pictures of text** (scanned PDFs). Tesseract + Poppler, which must be **on the OS**. Slow (7 pages ≈ 2 min) and makes mistakes.
🔧 Bake it into the indexing job's Docker image.

Managed parser: Textract, Azure Document Intelligence, LlamaParse. Paid, for hard layouts.

③ Chunking

Text splitter: cuts Documents into chunks about **one topic** each. A chunk is what the LLM reads.

Recursive splitter: tries paragraph → line → space, so it cuts at natural breaks. Handbook: **8 pages** → **16 chunks** (900 size).

Chunk overlap: neighbours share ~120 characters, so a sentence at a cut isn't lost.
🔧 Like overlapping maintenance windows.

Token: the word-piece the model reads and **bills**. ~4 characters each. Encodings r50k / cl100k / o200k count differently.

④ Safe storage

Stable chunk ID: `uuid5(filename + sha1(text))`. The same text always gets the same ID, so a re-run doesn't duplicate.
🔧 Idempotent like `kubectl apply`; content hash like an image digest.

PII redaction: mask names, phones, Aadhaar **before** embedding. Vectors end up in backups too!

Multi-tenancy: `tenant_id` in the payload + a **filter on every search**.
🔧 Namespaces + RBAC, a mandatory WHERE `tenant_id=?`

Search & answer words

From vectors to a grounded answer

⑤ Embeddings & store

embed_documents / embed_query: the two methods **every** embedding class has (many texts / one question).

OpenAI vs HuggingFace/Ollama: hosted (pay per call) vs **local** (free, data never leaves).
🔧 SaaS vs self-hosted: a privacy and cost decision.

QdrantVectorStore: LangChain's Qdrant wrapper. `from_documents` = embed + create + upsert in one call.
🔧 Like `kubectl run`; creating the collection yourself is the reviewed manifest.

add_documents / similarity_search: write (indexing) / read (top-k).

401 "User not found": the API key wasn't loaded. **Restart the kernel**, then `load_dotenv()`.

⑥ Retrieve

Retriever: the search wrapped as a Runnable: `.invoke(q)` → `Documents`. `k=4` = how many.

Always returns k: even if the right doc isn't indexed, you get the "closest" junk. Check coverage!

⑦ Generate

ChatOpenRouter: LangChain's chat model for OpenRouter. Returns an **AIMessage** (text in `.content`).

Temperature: a creativity dial. **0** = same answer every time (RAG, help desk). 0.7+ = creative.
🔧 0 = a pinned, deterministic config.

System / human message: system = the bot's rules, human = the user's question.

Prompt template: a reusable prompt with `{context}` and `{question}` slots.
🔧 A Helm/Jinja template: fixed structure, values per request.

Grounding rules: "answer only from context · say you don't know · never invent amounts".

format_docs: turns Documents into one numbered text block with filenames, so the bot can cite.

⑧ Chain it

LCEL: LangChain Expression Language. Join Runnables with **|** and get .invoke / .stream / .batch free.

RunnablePassthrough: passes the question through unchanged, while the other branch builds the context.
🔧 Like `tee`: same input, two paths.

Output parser: `StrOutputParser` turns an AIMessage into a plain string.

Two-step vs agentic RAG: retrieve once then answer, vs an **agent** deciding when and what to search (coming later).

Before the interview

Read this page last. Say it out loud once.

"How would you build RAG with LangChain?"

"**Indexing job:** loaders turn PDFs, wiki and CSVs into Documents with metadata. OCR handles scans. The recursive splitter makes ~900-character chunks with overlap. Each chunk gets a deterministic ID, PII is redacted, and a tenant_id is added. Then **embed + upsert into Qdrant**. **Query path:** an LCEL chain, retriever (top-4, tenant filter) → prompt with grounding rules → LLM at temperature 0 → string parser. If an answer is wrong, I debug stage by stage, starting with retrieval."

Quick answers

Why LangChain?

One interface per job, swappable vendors, ready loaders, | pipelines with streaming and batching.

What's a Document?

page_content + metadata. Metadata → payload → filters + citations.

Scanned PDF?

OCR (Tesseract + Poppler on the OS), slow, so spot-check it. Or Textract / Azure.

No duplicates on re-index?

Content-hash IDs (sha1 + uuid5) + upsert. Idempotent.

Compliance?

Redact PII first, skip ID scans, tenant_id + mandatory filter.

Quantization?

int8 = 4× less RAM, binary = 32×, for a small accuracy loss.

Vector store vs retriever?

Store = save + search. Retriever = search as a pluggable Runnable.

Temperature for RAG?

0: factual and repeatable.

What's LCEL?

The Runnable | pipe: output of the left → input of the right.

Local vs hosted embeddings?

Local (HF/Ollama) = private and free. Hosted = no infra. A business call.

The lab in numbers

Handbook	8 pages · 11,016 chars
Chunks	16 (900 / 120)
Embedding dims	1536 (3-small)
Retriever k	4 → 3,084 chars
OCR scan	56 elements · 1m54s

Errors seen in class

No module 'unstructured'	uv add unstructured
import markdown fails	uv add markdown
zsh: no matches [...]	quote "unstructured[pdf]"
401 User not found	restart kernel + .env

Chapter 02 ✓